



**TRƯỜNG ĐẠI HỌC TRÀ VINH
KHOA KỸ THUẬT VÀ CÔNG NGHỆ**

ĐỒ ÁN MÔN HỌC

Đề tài: HỆ THỐNG QUẢN LÝ DỰ ÁN

**Giảng viên hướng dẫn:
Nguyễn Bảo Ân**

**Nhóm sinh viên thực hiện: Nhóm 05
Họ tên: Nguyễn Thị Huyền Trân
MSSV:110122190
Họ tên: Nguyễn Kế Tông
MSSV:110122187
Họ tên: Hồ Quang Vinh
MSSV:110122201
Mã lớp: DA22TTC**

HỆ THỐNG QUẢN LÝ DỰ ÁN

Chương 1: Giới thiệu dự án

Chương 2 : Cơ sở lý thuyết

Chương 3:
Thiết kế hệ thống

Chương 4:
Triển khai và quản lý dự án

Chương 5:
Kiểm thử

Chương 6:
kết quả chương trình

Chương 7 : Kết luận & Hướng phát triển

Chương 1: Giới thiệu dự án

Trong thời đại số, việc quản lý dự án với nhiều thành viên và phòng ban trở nên phức tạp nếu không có hệ thống hỗ trợ hiệu quả. Các công cụ hiện tại như Jira, Trello,... tuy phổ biến nhưng có chi phí cao và giao diện phức tạp, chưa phù hợp với nhóm nhỏ hoặc môi trường giáo dục.

Do đó, nhóm quyết định xây dựng Hệ thống Quản lý Dự án – một nền tảng trực quan, dễ dùng, hỗ trợ lập kế hoạch, phân công, theo dõi tiến độ, và quản lý thành viên, dựa trên kiến trúc microservices, RESTful API, công nghệ cloud, CI/CD, Docker, và giao diện thân thiện.

Mục tiêu

Ứng dụng hướng đến việc hỗ trợ quản lý dự án hiệu quả và trực quan với các mục tiêu chính:

- Quản lý người dùng, phân quyền theo vai trò (Admin, PM, Leader, Member).
- Tạo/chỉnh sửa dự án, phân công và theo dõi nhiệm vụ.
- Thống kê, biểu đồ hóa tiến độ công việc.
- Giao diện đơn giản, dễ dùng, tương thích đa thiết bị.
- Ứng dụng các công nghệ hiện đại như: microservices, CI/CD, Docker.

Phạm vi và đối tượng sử dụng

Phạm vi:

- Đăng ký/đăng nhập, phân quyền theo vai trò.
- Tạo phòng ban, dự án, phân công nhiệm vụ.
- Theo dõi tiến độ và trạng thái công việc.
- Thống kê, hiển thị biểu đồ tiến độ.
- Chạy trên nền tảng web, hỗ trợ đa trình duyệt.

Đối tượng:

- Startup và nhóm phát triển phần mềm nhỏ.
- Doanh nghiệp vừa và nhỏ cần công cụ quản lý đơn giản, hiệu quả.

Chương 2: Cơ sở lý thuyết

Yêu cầu chức năng

Admin:

Quản lý người dùng, dự án, phòng ban, chức vụ, đội nhóm, danh mục công việc.

Theo dõi tiến độ tổng thể, quản lý thông báo.

PM (Project Manager):

Tạo, cập nhật tiến độ dự án.

Phân công công việc, đánh giá hiệu suất, báo cáo tiến độ.

Leader:

Chia nhiệm vụ, cập nhật tiến độ nhóm, theo dõi thông báo, hỗ trợ thành viên.

Member:

Xem và cập nhật công việc, ghi chú, đính kèm file, báo cáo tiến độ.

Yêu cầu phi chức năng

- Khả dụng: uptime > 99%, thông báo lỗi rõ ràng.
- Hiệu năng: phản hồi nhanh, hỗ trợ 100 người dùng đồng thời.
- Bảo mật: xác thực, phân quyền, mã hóa, chống tấn công.
- Thân thiện người dùng: giao diện đơn giản, đa nền tảng, hỗ trợ đa ngôn ngữ.
- Mở rộng: RESTful API, dễ tích hợp thêm tính năng.
- Kiểm thử & bảo trì: theo mô hình MVC, CI/CD.

Công nghệ phần mềm

Đặc điểm chính:

- An toàn: xác thực, phân quyền, mã hóa, chống tấn công.
- Riêng tư: thu thập tối thiểu, minh bạch, cho phép chỉnh sửa/xóa.
- Đảm bảo chất lượng: testing, code review, CI/CD.

Vòng đời phát triển:

- Phân tích yêu cầu
- Thiết kế hệ thống
- Lập trình
- Kiểm thử
- Triển khai
- Bảo trì

Mô hình Agile:

- Tư tưởng: linh hoạt, phản hồi nhanh, cải tiến liên tục.
- Thuật ngữ chính:
- Product Backlog, Sprint, Daily Stand-up, Retrospective, Burndown Chart.

Công nghệ sử dụng

TypeScript:

Mở rộng từ JavaScript, hỗ trợ kiểu tĩnh, hướng đối tượng, an toàn, dễ bảo trì.

Next.js:

Framework React hỗ trợ SSR, SSG, định tuyến tự động, API backend tích hợp.

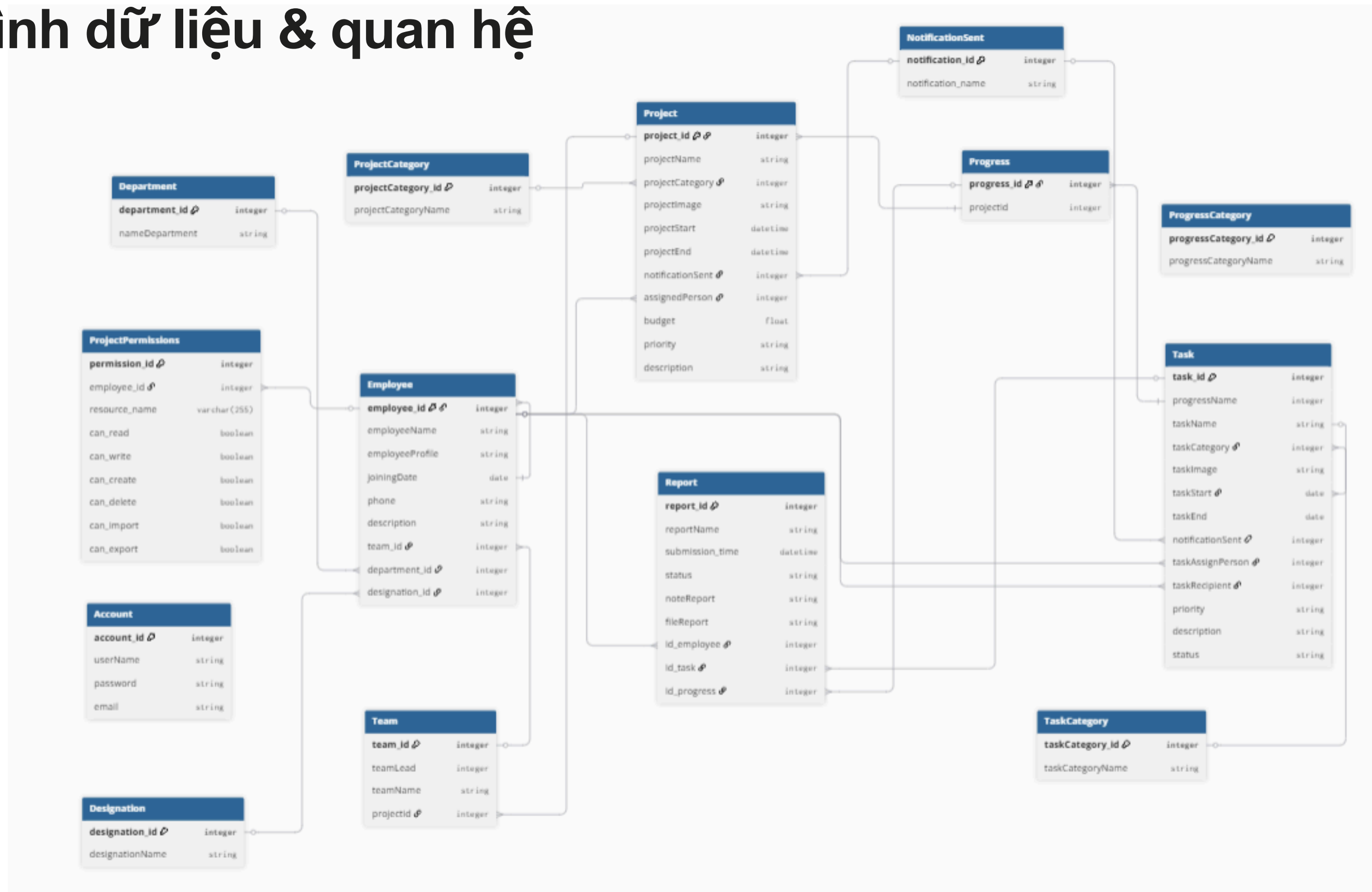
MongoDB:

Cơ sở dữ liệu NoSQL, lưu dữ liệu dạng JSON.

Hỗ trợ mở rộng, phân mảnh, bảo mật cao, tích hợp tốt với Node.js/NestJS.

Chương 3: THIẾT KẾ HỆ THỐNG

Mô hình dữ liệu & quan hệ



Thiết kế API

Mục tiêu

- Chuẩn hoá liên lạc giữa frontend & backend.
- Hỗ trợ đa nền tảng (Web, Mobile).
- Tái sử dụng, mở rộng, bảo mật.
- Theo chuẩn RESTful, trả dữ liệu JSON.

Các endpoint tiêu biểu

- POST /api/login: xác thực, trả token và user info.
- GET /api/employees: lấy danh sách nhân viên (có phân trang).
- POST /api/employees: thêm nhân viên mới.
- PUT /api/employees/{id}: cập nhật thông tin nhân viên.
- DELETE /api/employees/{id}: xóa nhân viên.
- GET /api/projects: lấy danh sách dự án (có thể lọc).
- POST /api/reports: tạo báo cáo mới, trả mã và trạng thái.

Thiết kế giao diện (UI/UX)

Mục tiêu & quy trình

- Giao diện thân thiện, dễ sử dụng.
- Mỗi vai trò người dùng có layout chức năng riêng phù hợp.
- Bố cục logic, nhất quán giữa các màn hình.

Quy trình thực hiện:

- Phân tích chức năng → xác định màn hình
- Phác thảo wireframe
- Thiết kế mockup
- Lấy phản hồi → chỉnh sửa
- Triển khai bằng React (Next.js)

Thiết kế giao diện (UI/UX)

Các giao diện chính

- /login – Đăng nhập
- /dashboard – Trang tổng quan
- /profile/bio – Hồ sơ cá nhân
- /departments, /designations, /teams, /employees, /notifications, /accounts, /projectcategories, /progresscategories, /taskcategories, /projects

CHƯƠNG 4 – TRIỂN KHAI & QUẢN LÝ DỰ ÁN

CI/CD (Triển khai liên tục)

Mục tiêu:

- Tự động kiểm tra lỗi, test, build & deploy ứng dụng.
- Đảm bảo chất lượng, phát hiện lỗi sớm, rút ngắn thời gian phát triển.

Công cụ:

- GitHub: Lưu trữ mã + chạy workflow CI/CD.
- Vercel: Triển khai frontend.
- Render: Triển khai backend.
- Docker: Đóng gói ứng dụng.
- MongoDB: Cơ sở dữ liệu.

CI/CD (Triển khai liên tục)

Mục tiêu:

- Tự động kiểm tra lỗi, test, build & deploy ứng dụng.
- Đảm bảo chất lượng, phát hiện lỗi sớm, rút ngắn thời gian phát triển.

Công cụ:

- GitHub: Lưu trữ mã + chạy workflow CI/CD.
- Vercel: Triển khai frontend.
- Render: Triển khai backend.
- Docker: Đóng gói ứng dụng.
- MongoDB: Cơ sở dữ liệu.

Quy trình CI/CD:

- Push code lên GitHub.
- GitHub Actions: Lint, build, test tự động.
- Nếu thành công → Vercel/Render tự động deploy.
- GitHub thông báo kết quả từng bước.

Docker & Triển khai thực tế

Docker là gì?

- Đóng gói ứng dụng vào container (nhẹ, độc lập).
- Giảm lỗi môi trường giữa dev và prod.

Mục đích sử dụng:

- Di động & tái sử dụng.
- Dễ bảo trì/nâng cấp (tách FE, BE, DB).
- Triển khai nhanh.
- Hỗ trợ teamwork đồng bộ môi trường.

Dockerfile & Docker Compose

- Dockerfile: Tạo image cho frontend (Next.js), backend (NestJS).
- Docker Compose: Chạy đồng thời nhiều container (FE + BE + DB) → chỉ cần 1 lệnh để khởi chạy toàn hệ thống.

Quản lý dự án với Jira

Jira là gì?

- Công cụ quản lý dự án Agile (Scrum, Kanban).
- Theo dõi tiến độ, backlog, sprint, task.

Ưu điểm:

- Giao diện trực quan.
- Hỗ trợ Burndown Chart, phân quyền.
- Tích hợp GitHub, Slack...

Chức năng Jira

- Quản lý backlog: Tạo, sắp xếp, đánh độ ưu tiên task.
- Lập kế hoạch Sprint: Chọn task, đặt thời gian.
- Phân công task: Theo dõi tiến độ từng thành viên.
- Báo cáo & giao tiếp nhóm: Burndown Chart, comment, log hoạt động.

Quy trình Jira trong dự án

- Tạo project: Template Scrum.
- Xây dựng backlog: Epic, Story, Task.
- Lập Sprint:
- Sprint 1: Giao diện cơ bản.
- Sprint 2: API & chức năng chính.
- Sprint 3: Kiểm thử, tối ưu.
- Theo dõi bằng Kanban Board: To Do → In Progress → Done.
- Burndown Chart: Theo dõi khối lượng công việc còn lại.
- Tổng kết Sprint: Review, lý do trễ, lên kế hoạch tiếp theo.

The image shows two screenshots of the Jira interface. The top screenshot displays a Scrum Backlog with three items:

Item	Status	Progress	Assignee
☒ SCRUM-4 Thiết kế UI (Login, Dashboard - Figma)	TO DO	-	[User Icon]
☐ SCRUM-5 Thiết kế cơ sở dữ liệu (ERD), tạo project NestJS	TO DO	-	[User Icon]
☐ SCRUM-6 Tạo Jira project, viết chương 1-2 báo cáo	TO DO	-	[User Icon]

Below the items is a '+ Create' button.

The bottom screenshot shows 'SCRUM Sprint 1' running from 9 Jun to 22 Jun with 3 work items:

Item	Status	Progress	Assignee
☒ SCRUM-28 Giao diện Login/Register (Next.js + Tailwind)	TO DO	-	[User Icon]
☒ SCRUM-29 API Login/Register, CRUD User + phân quyền	TO DO	-	[User Icon]

At the top of the sprint view, there are counters for '0' completed, '0' in progress, and '0' remaining items, along with a 'Start sprint' button and a menu icon.

CHƯƠNG 5: KIỂM THỬ

Mục đích kiểm thử

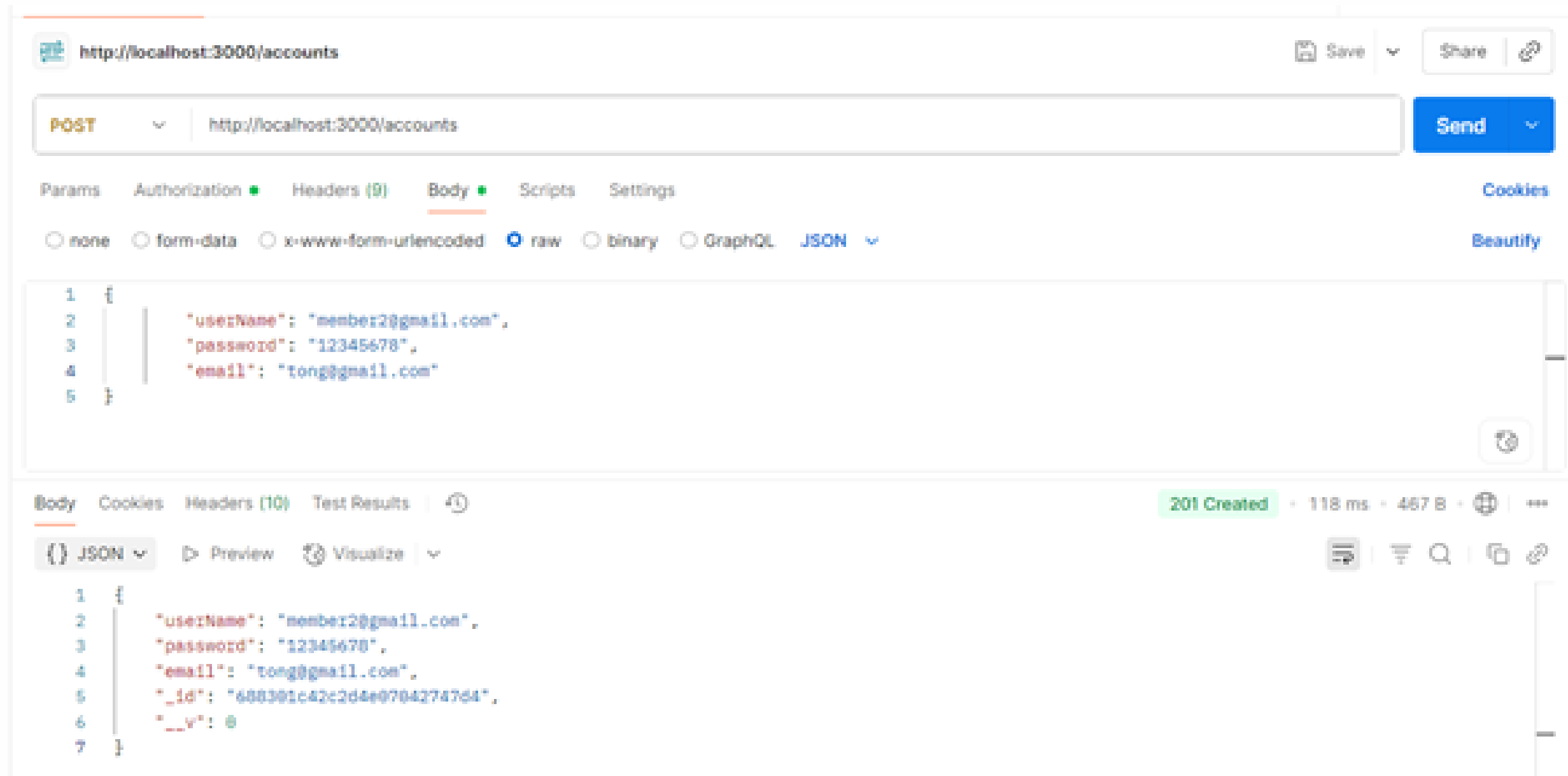
- Đảm bảo hệ thống hoạt động đúng yêu cầu.
- Phát hiện lỗi logic, nhập liệu, sai luồng xử lý.
- Kiểm tra tính ổn định, giao tiếp giữa frontend – backend – DB.
- Cải thiện trải nghiệm người dùng (UI/UX).

Công cụ sử dụng

- Postman: Kiểm thử API (GET, POST, PUT, DELETE).
- Trình duyệt (Chrome, Firefox): Kiểm tra giao diện, bố cục, tính phản hồi.

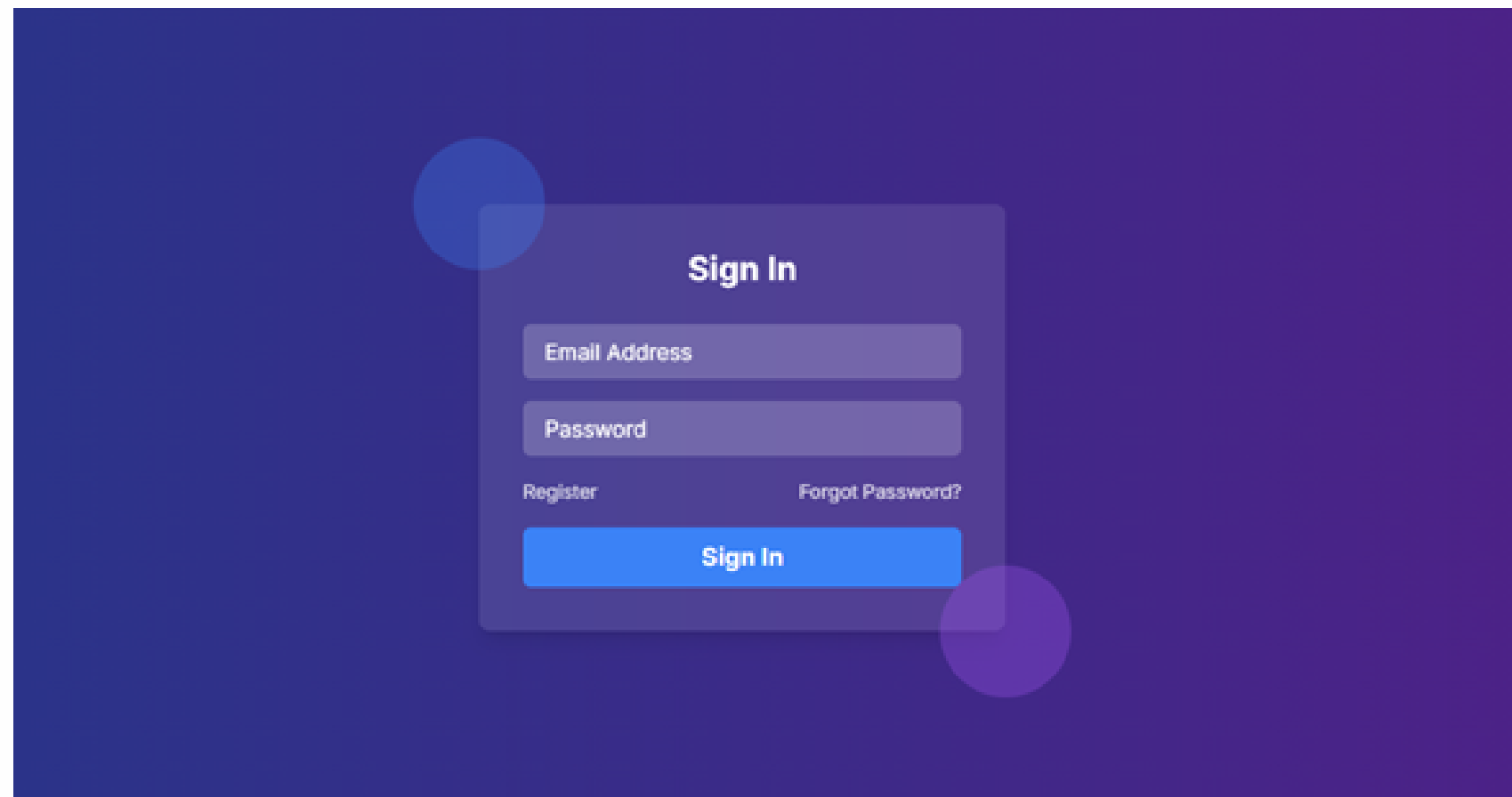
Kết quả kiểm thử

- Thực hiện kiểm thử các chức năng: tạo, sửa, tìm account.
- Hệ thống phản hồi đúng, đầy đủ dữ liệu.
- Không phát hiện lỗi nghiêm trọng trong quá trình kiểm thử.

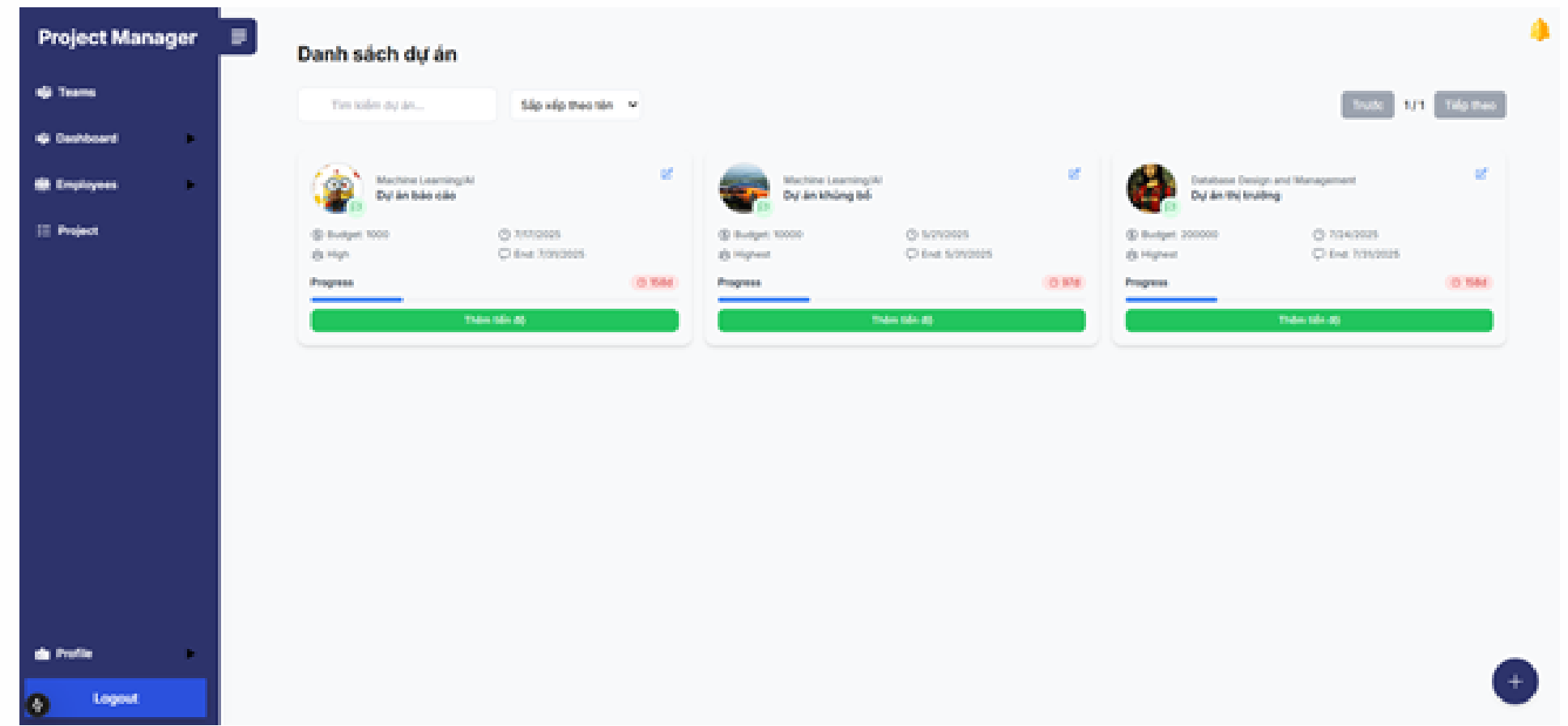


CHƯƠNG 6 – KẾT QUẢ CHƯƠNG TRÌNH

Giao diện Đăng nhập



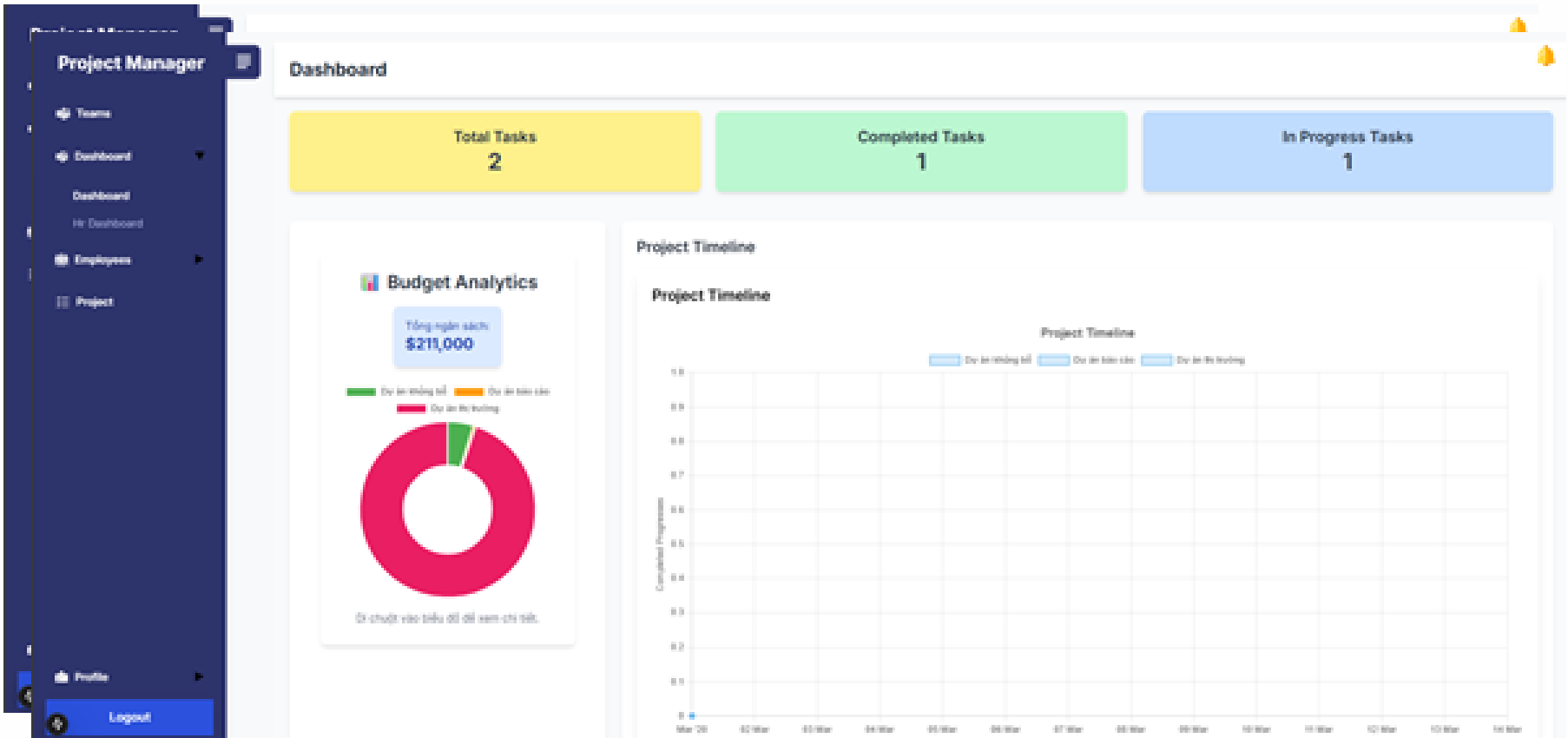
Giao diện role Manager



HR Dashboard

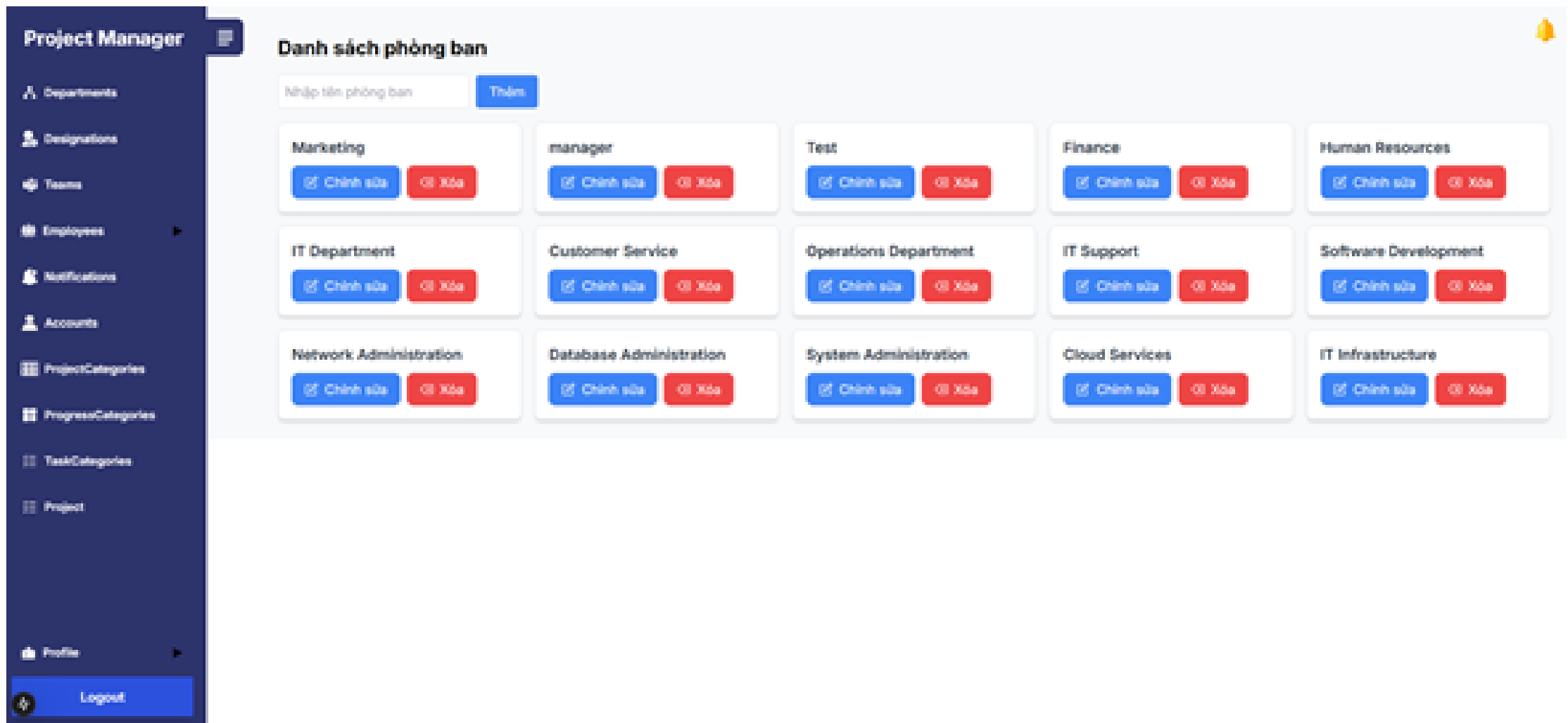


Project Dashboard



Giao diện ROLE ADMIN

Danh sách Department



CHƯƠNG 7 – ĐÁNH GIÁ VÀ KẾT LUẬN

Những Khó Khăn Gặp Phải

Khó thiết kế hệ thống phân quyền (Admin, PM, Leader, Member).

Lỗi khi tích hợp frontend & backend (API, token, JSON).

Chưa thành thạo Docker (cấu hình sai port, volume).

Hạn chế từ dịch vụ miễn phí (Render, Vercel).

GitHub Actions cần học cú pháp YAML, tốn thời gian debug.

Bài Học Rút Ra

Kỹ năng teamwork cải thiện nhờ Jira và phân công rõ ràng.

Hiểu rõ CI/CD và Docker qua trải nghiệm thực tế.

Tự chủ trong nghiên cứu và tra cứu tài liệu kỹ thuật.

Nâng cao năng lực lập trình fullstack: Next.js, NestJS, MongoDB, REST API.

CHƯƠNG 7 – ĐÁNH GIÁ VÀ KẾT LUẬN

Hướng phát triển

- Phát triển mobile app cho iOS và Android bằng React Native hoặc Flutter.
- Tối ưu giao diện người dùng (UI/UX): thiết kế hiện đại, thân thiện trên đa nền tảng.
- Tích hợp AI: phân tích tiến độ dự án, gợi ý phân công task tự động.
- Kết nối với các công cụ thứ ba như Google Calendar, Slack, Notion để đồng bộ công việc.
- Tăng tính bảo mật: mã hóa dữ liệu, xác thực 2 lớp (2FA), kiểm soát truy cập chặt chẽ.
- Quy mô hệ thống: triển khai trên Kubernetes để đảm bảo khả năng mở rộng (scalability) khi có nhiều người dùng.
- Thống kê – báo cáo tự động: biểu đồ Gantt, burn down chart, hiệu suất từng thành viên.